WebStarter CLI v2.0 🚀

Smart PHP Project Generator - Instantly scaffold production-ready PHP applications with authentication, admin panel, PHPMailer, and Phosphor Icons.

Built with ❤️ for rapid PHP development.

✨ Features

- 🏗️ **Smart Folder Structure -** Organized like lymora.tech
- 🔐 **Authentication System -** Login, Register, Logout with session management
- 👨‍💼 **Admin Panel -** Optional admin dashboard
- 📧 **PHPMailer -** Automatic installation via Composer
- 🎨 **Phosphor Icons -** Beautiful icon library included
- 🗄️ **Database Schema -** Pre-configured MySQL tables
- 🎯 **Clean URLs -** .htaccess routing configured
- 📝 **Environment Config -** .env file support
- 🔒 **Security -** Best practices baked in
- 📱 **Responsive CSS -** Mobile-first styling

📦 Installation

Prerequisites

- **Node.js** v14 or higher
- **PHP** 7.4 or higher
- **Composer** (Download here)
- **MySQL** database

Install Globally

```bash
npm install -g webstarter-cli
```

Or run directly with `npx`:

```bash
npx webstarter-cli
```

🚀 Quick Start

1. Create a New Project

```bash
webstarter
```

Or use the alias:

```bash
create-webstarter
```

2. Answer the Prompts

```
✨ Your name: John Doe
📁 Project name: my-awesome-app
🔐 Include authentication? Yes
👨‍💼 Include admin panel? No
📧 Install PHPMailer? Yes
🎨 Download Phosphor Icons? Yes
```

3. Navigate to Your Project

```bash
cd my-awesome-app
```

4. Setup Database

```bash
mysql -u root -p < dbschema/database.sql
```

5. Configure Environment

Edit the `.env` file with your database credentials:

```env
DB_HOST=localhost
DB_NAME=my_awesome_app
DB_USER=root
DB_PASS=your_password
```

6. Start Development Server

```bash
php -S localhost:8000
```

Visit: `http://localhost:8000`

📁 Project Structure

```
my-awesome-app/
├── api/                    # Backend API endpoints
│   ├── auth/               # Authentication (login, register, logout)
│   ├── dashboard/          # Dashboard APIs
│   ├── payments/           # Payment processing
│   ├── utilities/          # Helper utilities
│   └── webhooks/           # Webhook handlers
│
├── assets/                 # Static assets
│   ├── css/                # Stylesheets
│   │   ├── main.css        # Main styles
│   │   └── responsive.css  # Responsive design
│   ├── js/                 # JavaScript files
│   ├── icons/              # Phosphor Icons (SVG)
│   ├── images/             # Image assets
│   ├── fonts/              # Web fonts
│   └── favicon/            # Favicon files
│
├── config/                 # Configuration files
│   ├── constants.php       # App constants
│   ├── env.php             # Environment loader
│   └── database.php        # Database connection
│
├── pages/                  # Frontend pages
```

```
│   ├── public/            # Public pages
│   │   ├── index.php      # Homepage
│   │   ├── login.php      # Login page
│   │   └── register.php   # Register page
│   ├── user/              # User dashboard
│   │   └── dashboard.php  # User dashboard
│   └── admin/             # Admin panel (optional)
│       └── dashboard.php  # Admin dashboard
│
├── dbschema/              # Database schemas
│   └── database.sql       # MySQL schema
│
├── uploads/               # User uploads
├── vendor/                # Composer dependencies
├── .env                   # Environment variables
├── .htaccess              # Clean URL routing
├── index.php              # Application entry point
└── README.md              # Project documentation
```

🔐 Authentication System

API Endpoints

Login

```bash
POST /api/auth/login.php
Content-Type: application/json

{
  "email": "user@example.com",
  "password": "password123"
}
```

Register

```bash
```

```
POST /api/auth/register.php
Content-Type: application/json

{
  "email": "user@example.com",
  "password": "password123",
  "full_name": "John Doe"
}
```

Logout

```bash
GET /api/auth/logout.php
```

Frontend Pages

- **Login:** `/pages/public/login.php`
- **Register:** `/pages/public/register.php`
- **User Dashboard:** `/pages/user/dashboard.php`
- **Admin Dashboard:** `/pages/admin/dashboard.php` (if enabled)

🎨 Using Phosphor Icons

Phosphor Icons are automatically downloaded to `assets/icons/`.

Usage Example

```html
<img src="/assets/icons/core-main/assets/regular/user-circle.svg" alt="User">
```

Or in PHP:

```php
<img src="<?php echo '/assets/icons/core-main/assets/bold/bell.svg'; ?>" alt="Notifications">
```

📧 PHPMailer Setup

PHPMailer is automatically installed via Composer.

Configure SMTP

Edit `.env`:

```env
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your-email@gmail.com
SMTP_PASS=your-app-password
SMTP_FROM=noreply@yourapp.com
```

Send Email

```php
```

```php
<?php
require_once __DIR__ . '/vendor/autoload.php';

use PHPMailer\PHPMailer\PHPMailer;

$mail = new PHPMailer(true);

try {
    $mail->isSMTP();
    $mail->Host = getenv('SMTP_HOST');
    $mail->SMTPAuth = true;
    $mail->Username = getenv('SMTP_USER');
    $mail->Password = getenv('SMTP_PASS');
    $mail->SMTPSecure = PHPMailer::ENCRYPTION_STARTTLS;
    $mail->Port = getenv('SMTP_PORT');

    $mail->setFrom(getenv('SMTP_FROM'), getenv('SMTP_FROM_NAME'));
    $mail->addAddress('recipient@example.com', 'Recipient Name');

    $mail->isHTML(true);
    $mail->Subject = 'Welcome!';
    $mail->Body = '<h1>Hello World</h1>';

    $mail->send();
    echo 'Email sent!';
} catch (Exception $e) {
    echo "Email failed: {$mail->ErrorInfo}";
}
?>
```

🗄️ Database Schema

The CLI generates a complete MySQL schema with:

- **users** table - User accounts with email, password, role
- **password_resets** table - Password reset tokens
- **sessions** table - User session tracking

Import Schema

```bash
```

```
mysql -u root -p < dbschema/database.sql
```

⚙️ Configuration

Environment Variables

Edit `.env`:

```env
# Database
DB_HOST=localhost
DB_NAME=your_database
DB_USER=root
DB_PASS=password

# Application
APP_NAME="My Awesome App"
APP_URL=http://localhost
APP_ENV=development

# Security
SESSION_LIFETIME=3600
PASSWORD_HASH_COST=12

# Email
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your-email@gmail.com
SMTP_PASS=your-app-password
```

Database Connection

The database class uses singleton pattern:

```php
```

```php
<?php
$db = Database::getInstance()->getConnection();

// Example query
$stmt = $db->prepare("SELECT * FROM users WHERE id = :id");
$stmt->execute(['id' => 1]);
$user = $stmt->fetch();
```

🔒 Security Features

- ✅ Password hashing with `password_hash()`
- ✅ PDO prepared statements (SQL injection protection)
- ✅ Session management
- ✅ CSRF protection ready
- ✅ Environment variable protection
- ✅ .htaccess security headers
- ✅ Sensitive file access blocking

🛠️ Development

Local Development Server

```bash
php -S localhost:8000
```

Production Deployment

1. **Set production environment:**

```env
APP_ENV=production
```

2. **Uncomment HTTPS redirect in .htaccess:**

```apache
```

```
RewriteCond %{HTTPS} off
RewriteRule ^(.*)$ https://%{HTTP_HOST}%{REQUEST_URI} [L,R=301]
```

3. **Configure proper database credentials**
4. **Set proper file permissions:**

```bash
chmod 755 uploads/
chmod 644 .env
```

🔧 Troubleshooting

Composer Not Found

If Composer is not found automatically:

1. Download from getcomposer.org
2. When prompted, provide the full path to `composer.exe`
3. The CLI will save this path for future use

Database Connection Error

Check your `.env` file:

- Verify `DB_HOST`, `DB_NAME`, `DB_USER`, `DB_PASS`
- Ensure MySQL is running
- Confirm database exists

Icons Not Downloading

If automatic download fails:

1. Visit phosphoricons.com
2. Download icons manually
3. Extract to `assets/icons/`

📚 Commands

Global Installation

```bash
npm install -g webstarter-cli
```

Run Without Installing

```bash
npx webstarter-cli
```

Create Project

```bash
webstarter
# or
create-webstarter
```

🤝 Contributing

Contributions welcome! Feel free to submit issues and pull requests.

📄 License

MIT License - Feel free to use for personal and commercial projects.

🙏 Credits

- **Phosphor Icons:** phosphoricons.com
- **PHPMailer:** github.com/PHPMailer/PHPMailer
- **Composer:** getcomposer.org

💡 Tips

1. **Use environment variables -** Never commit `.env` to version control
2. **Follow PSR standards -** Keep code clean and consistent
3. **Test thoroughly -** Especially authentication flows
4. **Keep dependencies updated -** Run `composer update` regularly
5. **Use prepared statements -** Always protect against SQL injection

---

Made with ❤️ for the PHP community

🌟 Star this repo if you find it helpful!